

Tarea UT5.- Diseño orientado a objetos. Elaboración de diagramas estructurales.

Abraham Quintana Micó
1 DAW semi B

Tarea 1: Elabora un diagrama de clases para una empresa dedicada al alquiler de automóviles, teniendo en cuenta los siguientes requisitos:

- Un determinado cliente puede tener, en un momento dado, varias reservas hechas.
- De cada cliente, se desea almacenar su DNI, nombre, dirección y teléfono.
- Cada cliente puede ser avalado por otro cliente de la empresa. Un cliente puede avalar a varios.
- Una reserva la realiza un único cliente, pero puede involucrar varios coches.
- Es importante registrar la fecha de inicio y fin de la reserva, el precio de alquiler de cada uno de los coches, los litros de gasolina en el momento de realizar la reserva, el precio total de la reserva y un indicador que informe acerca de si cada coche ha sido entregado o no.
- Todo coche tiene siempre asignado un determinado garaje. De cada coche, se requiere registrar su matrícula, modelo, color y marca.
- Sobre cada garaje, es necesario conocer su código único, ubicación, capacidad o número máximo de coches que caben en el garaje.

• Crear la tabla de sustantivos, seleccionar las clases.

Clase/objeto potencial	Categoría
Cliente	Clase
DNI	Atributo clave primaria del cliente
Nombre	Atributo
Dirección	Atributo
Teléfono	Atributo
Aval	Atributo que representa un elemento de la clase Cliente
Reservas	Atributo listado de elementos de la clase Reserva
Reserva	Clase
Cliente	Atributo que hace referencia a un único cliente
FechaInicio	Atributo representa el inicio de la reserva. (entiendo por los requisitos que todos los coches de una reserva se entregan el mismo día)
FechaFin	Atributo representa el fin de la reserva. (entiendo por los requisitos que todos los coches de una reserva se devuelven el mismo día)
PrecioTotal	Atributo que representa la suma de los precios de todos los coches reservados.
Coches	Atributo listado de elementos de la clase Coche
Entregados	Atributo que define si todos los coches han sido entregados
Reserva_Coche	Clase
Coche	Atributo que representa un elemento de la clase Coche
Reserva	Atributo que representa un elemento de la clase Reserva
Precio	Atributo
CombustibleInicial	Atributo
Entregado	Atributo
Coche	Clase
Matrícula	Atributo clave primaria
Modelo	Atributo
Color	Atributo
Marca	Atributo
Garaje	Atributo garaje asignado, que enlaza con un garaje.
Garaje	Clase
Código	Atributo clave primaria
Ubicación	Atributo
Capacidad	Atributo

- **Obtención de las relaciones.**

Cliente:

- Aval 1 Se Relaciona con otro elemento de tipo Cliente mediante el atributo DNI del cliente que avala, puede ser null si no tiene avalador.
- Reservas 1..* Se relaciona con uno o varios elementos de tipo Reserva

Reserva:

- Cliente 1 Se relaciona con un elemento de tipo Cliente mediante el atributo DNI del cliente.
- Coches N..M Se relaciona con uno o varios elementos del tipo Coche, a su vez un coche puede estar en varias reservas.

Reserva_Coche:

- Reserva 1 Se relaciona con un elemento de tipo Reserva mediante el atributo Id_Reserva del elemento Reserva
- Coche 1 Se relaciona con un elemento de tipo Coche mediante el atributo Matrícula del elemento Coche.

Coche:

- Garaje 1 Se relaciona con un elemento de tipo Garaje mediante el atributo Código del elemento Garaje

Garaje:

- Coches 0..* Se relaciona varios elementos de tipo Coche mediante el atributo Matricula del elemento Coche, puede ser null si no tiene ningún coche asignado.

- **Obtener los atributos.**

Cliente:

- DNI: String
- Nombre: String
- Dirección: String
- Teléfono: Int
- Aval: String
- Reservas (Listado de elementos de la clase Reserva): List<Int>

Reserva:

- Id_Reserva: Int
- Cliente: String
- Fecha_Inicio: Date
- Fecha_Fin: Date
- Precio_Total: Int
- Coches (Listado de coches) List<String>
- Entregados: Boolean

Reserva_Coche:

- Reserva: Int
- Coche: String
- Precio: Double
- CombustibleInicial: Int
- Entregado: Boolean

Coche:

- Matrícula: String
- Modelo: String
- Color: String
- Marca: String
- Garaje: Int

Garaje:

- Código: Int
- Ubicación: String
- Capacidad: Int

- **Añadir getters, setters y constructores.**

Clase Cliente:

Método	Descripción
Constructor	Cliente(String dni, String nombre, String direccion, int telefono, String aval, List<Integer> reservas)
getDni()	Devuelve el DNI del cliente.
setDni(String dni)	Establece el DNI del cliente.
getNombre()	Devuelve el nombre del cliente.
setNombre(String nombre)	Establece el nombre del cliente.
getDireccion()	Devuelve la dirección del cliente.
setDireccion(String direccion)	Establece la dirección del cliente.
getTelefono()	Devuelve el teléfono del cliente.
setTelefono(int telefono)	Establece el teléfono del cliente.
getAval()	Devuelve el aval del cliente.
setAval(String aval)	Establece el aval del cliente.
getReservas()	Devuelve la lista de reservas del cliente.
setReservas(List<Integer> reservas)	Establece la lista de reservas del cliente.

Clase Reserva(El Id debe ser autogenerado y único):

Método	Descripción
Constructor	Reserva(String cliente, Date fechaInicio, Date fechaFin, int precioTotal, List<String> coches, boolean entregados)
getCliente()	Devuelve el cliente de la reserva.
setCliente(String cliente)	Establece el cliente de la reserva.
getFechaInicio()	Devuelve la fecha de inicio de la reserva.
setFechaInicio(Date fechaInicio)	Establece la fecha de inicio de la reserva.
getFechaFin()	Devuelve la fecha de fin de la reserva.
setFechaFin(Date fechaFin)	Establece la fecha de fin de la reserva.
getPrecioTotal()	Devuelve el precio total de la reserva.
setPrecioTotal(int precioTotal)	Establece el precio total de la reserva.
getCoches()	Devuelve la lista de coches de la reserva.
setCoches(List<String> coches)	Establece la lista de coches de la reserva.
isEntregados()	Devuelve si los coches han sido entregados.
setEntregados(boolean entregados)	Establece si los coches han sido entregados.

Clase Reserva_Coche:

Método	Descripción
Constructor	Reserva_Coche(int reserva, String coche, double precio, int combustibleInicial, boolean entregado)
getReserva()	Devuelve el ID de la reserva.
setReserva(int reserva)	Establece el ID de la reserva.
getCoche()	Devuelve la matrícula del coche.
setCoche(String coche)	Establece la matrícula del coche.
getPrecio()	Devuelve el precio del alquiler del coche.
setPrecio(double precio)	Establece el precio del alquiler del coche.
getCombustibleInicial()	Devuelve el combustible inicial del coche.
setCombustibleInicial(int combustibleInicial)	Establece el combustible inicial del coche.
isEntregado()	Devuelve si el coche ha sido entregado.
setEntregado(boolean entregado)	Establece si el coche ha sido entregado.

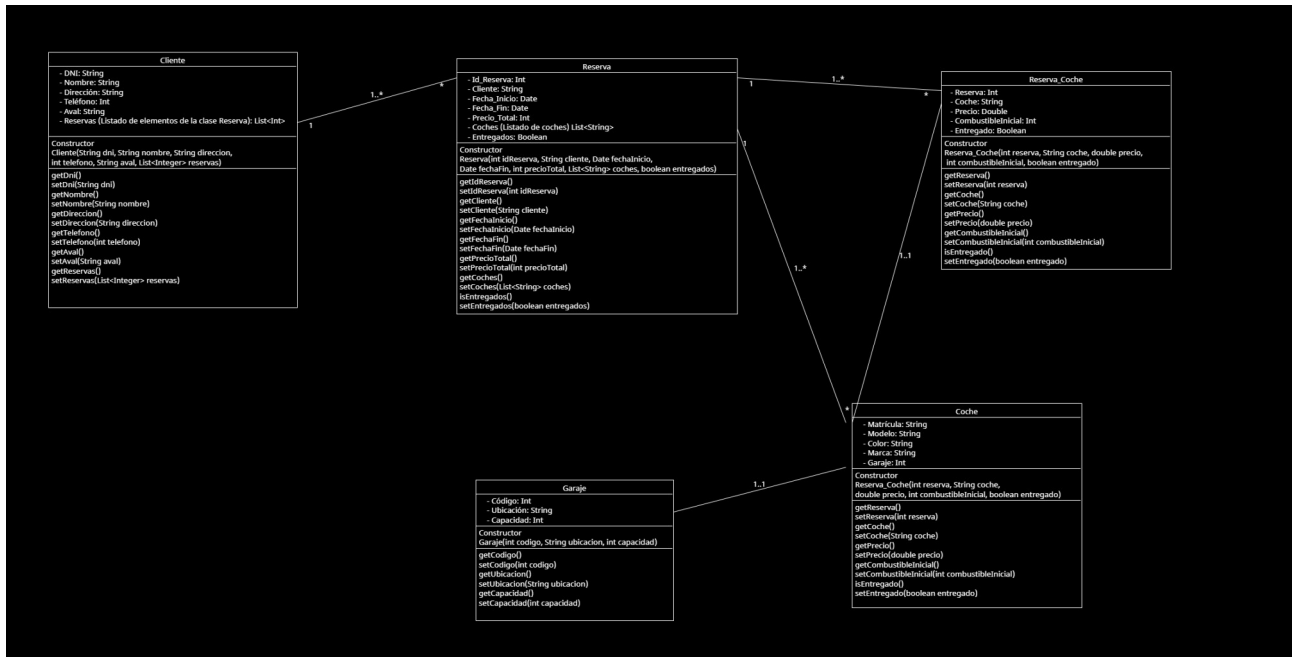
Clase Coche:

Método	Descripción
Constructor	Coche(String matricula, String modelo, String color, String marca, int garaje)
getMatricula()	Devuelve la matrícula del coche.
setMatricula(String matricula)	Establece la matrícula del coche.
getModelo()	Devuelve el modelo del coche.
setModelo(String modelo)	Establece el modelo del coche.
getColor()	Devuelve el color del coche.
setColor(String color)	Establece el color del coche.
getMarca()	Devuelve la marca del coche.
setMarca(String marca)	Establece la marca del coche.
getGaraje()	Devuelve el código del garaje donde está el coche.
setGaraje(int garaje)	Establece el código del garaje donde está el coche.

Clase Garaje:

Método	Descripción
Constructor	Garaje(int codigo, String ubicacion, int capacidad)
getCodigo()	Devuelve el código del garaje.
setCodigo(int codigo)	Establece el código del garaje.
getUbicacion()	Devuelve la ubicación del garaje.
setUbicacion(String ubicacion)	Establece la ubicación del garaje.
getCapacidad()	Devuelve la capacidad del garaje.
setCapacidad(int capacidad)	Establece la capacidad del garaje.

- Crear el diagrama.



Tarea 2: Elabora un diagrama de clases para un sistema de monitorización de sensores, teniendo en cuenta los siguientes requisitos:

- El sistema monitoriza sensores y también informa sobre posibles problemas.
- Cada sensor lo describe su fabricante y número de modelo, secuencia de inicialización (que se envía al sensor para iniciarlo), factor de escala y unidad de medida, intervalo de muestreo, ubicación, estado (encendido, apagado, en reserva), valor actual y umbral de alarma. Hay un tipo especial de sensores, que son los sensores críticos. Estos tienen una característica adicional, que es su tolerancia del intervalo de muestreo.
- Los sensores están instalados en edificios. El sistema hace un seguimiento de los sensores de cada edificio. De cada edificio, se conoce su dirección y el número de contacto en caso de emergencia.
- El sistema activa determinados dispositivos de alarma siempre que se supera el umbral de un sensor. Los dispositivos de alarma tienen dos características de las cuales depende su funcionamiento: el estado del dispositivo y la duración de la señal de alarma.
- El sistema registra información de la fecha, hora, gravedad, tiempo de reparación y estado de cada suceso de alarma.

• Crear la tabla de sustantivos, seleccionar las clases.

Clase/objeto potencial	Categoría
Sensor	Clase
IdSensor	Atributo
Fabricante	Atributo
Modelo	Atributo
SecuenciaInicio	Atributo
FactorEscala	Atributo
UnidadMedida	Atributo
IntervaloMuestreo	Atributo
Ubicacion	Atributo
Estado	Atributo
ValorActual	Atributo
UmbralAlarma	Atributo
ToleranciaIntervalo	Atributo
Edificio	Clase
Direccion	Atributo
NumeroEmergencia	Atributo
Alarma	Clase
IdAlarma	Atributo
Sensor	Atributo
Estado	Atributo
Duracion	Atributo
HistoricoAlarma	Clase
Alarma	Atributo
FechaYHora	Atributo
Gravedad	Atributo
TiempoReparacion	Atributo
Estado	Atributo

• Obtención de las relaciones.

Sensor:

- Ubicación 1 Se Relaciona con un elemento de tipo Edificio mediante el atributo dirección.
- Alarma 0..* Se relaciona con elementos de la clase Alarma, obteniendo las alarmas de un sensor mediante el atributo Sensor de la clase Alarma que debe coincidir con el IdSensor de esta clase.

Edificio:

- Sensor 0..* Se Relaciona con elementos de la clase Sensor mediante el atributo Ubicación de la clase Sensor que debe coincidir con el atributo Dirección esta clase, que debe ser único ya que no puede haber dos edificios en la misma dirección.

Alarma:

- Sensor 1 Se relaciona con un elemento de la clase Sensor mediante el atributo IdSensor de la clase Sensor, que debe con el atributo Sensor de esta clase.
- HistoricoAlarma 0..* Se relaciona mediante el atributo Alarma de la de la clase HistóricoAlarma, que debe coincidir con el IdAlarma de esta clase.

HistoricoAlarma:

- Alarma 1 Se relaciona mediante el atributo IdAlarma de la clase Alarma, que debe coincidir con el atributo Alarma de esta clase.

- **Obtener los atributos.**

Sensor

- IdSensor: int
- Fabricante: String
- Modelo: String
- SecuencialInicio: String
- FactorEscala: double
- UnidadMedida: String (puede ser una buena práctica, crear una clase UnidadMedidaEnum, para determinar que valores son aceptados)
- IntervaloMuestreo: int
- Ubicacion: String
- Estado: String (Determinado por un EstadoEnum que determine los tres valores posibles (encendido, apagado, en reserva))
- ValorActual: int
- UmbralAlarma: int
- ToleranciaIntervalo: int (null en el caso de sensores no críticos)

Edificio

- Direccion: String
- NumeroEmergencia: int

Alarma

- IdAlarma: int
- Sensor: int
- Estado: boolean
- Duracion: int

HistoricoAlarma

- Alarma: int
- FechaYHora: LocalDateTime
- Gravedad: int
- TiempoReparacion: int
- Estado: boolean

- **Añadir getters, setters y constructores.**

Clase Sensor

Método	Descripción
Sensor(int idSensor, String fabricante, String modelo, String secuencialInicio, double factorEscala, UnidadMedidaEnum unidadMedida, int intervaloMuestreo, String ubicacion, EstadoEnum estado, int valorActual, int umbralAlarma, Integer toleranciaIntervalo)	Constructor que inicializa todos los atributos de la clase Sensor.
getIdSensor()	Devuelve el ID del sensor.
setIdSensor(int idSensor)	Establece el ID del sensor.
getFabricante()	Devuelve el fabricante del sensor.
setFabricante(String fabricante)	Establece el fabricante del sensor.
getModelo()	Devuelve el modelo del sensor.
setModelo(String modelo)	Establece el modelo del sensor.
getSecuencialInicio()	Devuelve la secuencia de inicio del sensor.
setSecuencialInicio(String secuencialInicio)	Establece la secuencia de inicio del sensor.
getFactorEscala()	Devuelve el factor de escala del sensor.
setFactorEscala(double factorEscala)	Establece el factor de escala del sensor.
getUnidadMedida()	Devuelve la unidad de medida del sensor.
setUnidadMedida(UnidadMedidaEnum unidadMedida)	Establece la unidad de medida del sensor.
getIntervaloMuestreo()	Devuelve el intervalo de muestreo del sensor.
setIntervaloMuestreo(int intervaloMuestreo)	Establece el intervalo de muestreo del sensor.
getUbicacion()	Devuelve la ubicación del sensor.
setUbicacion(String ubicacion)	Establece la ubicación del sensor.
getEstado()	Devuelve el estado del sensor.
setEstado(EstadoEnum estado)	Establece el estado del sensor.
getValorActual()	Devuelve el valor actual del sensor.
setValorActual(int valorActual)	Establece el valor actual del sensor.
getUmbralAlarma()	Devuelve el umbral de alarma del sensor.
setUmbralAlarma(int umbralAlarma)	Establece el umbral de alarma del sensor.
getToleranciaIntervalo()	Devuelve la tolerancia del intervalo del sensor.
setToleranciaIntervalo(Optional<Integer> toleranciaIntervalo)	Establece la tolerancia del intervalo del sensor.

Clase Edificio:

Método	Descripción
Edificio(String direccion, int numeroEmergencia)	Constructor que inicializa la dirección y el número de emergencia del edificio.
getDireccion()	Devuelve la dirección del edificio.
setDireccion(String direccion)	Establece la dirección del edificio.
getNumeroEmergencia()	Devuelve el número de emergencia del edificio.
setNumeroEmergencia(int numeroEmergencia)	Establece el número de emergencia del edificio.

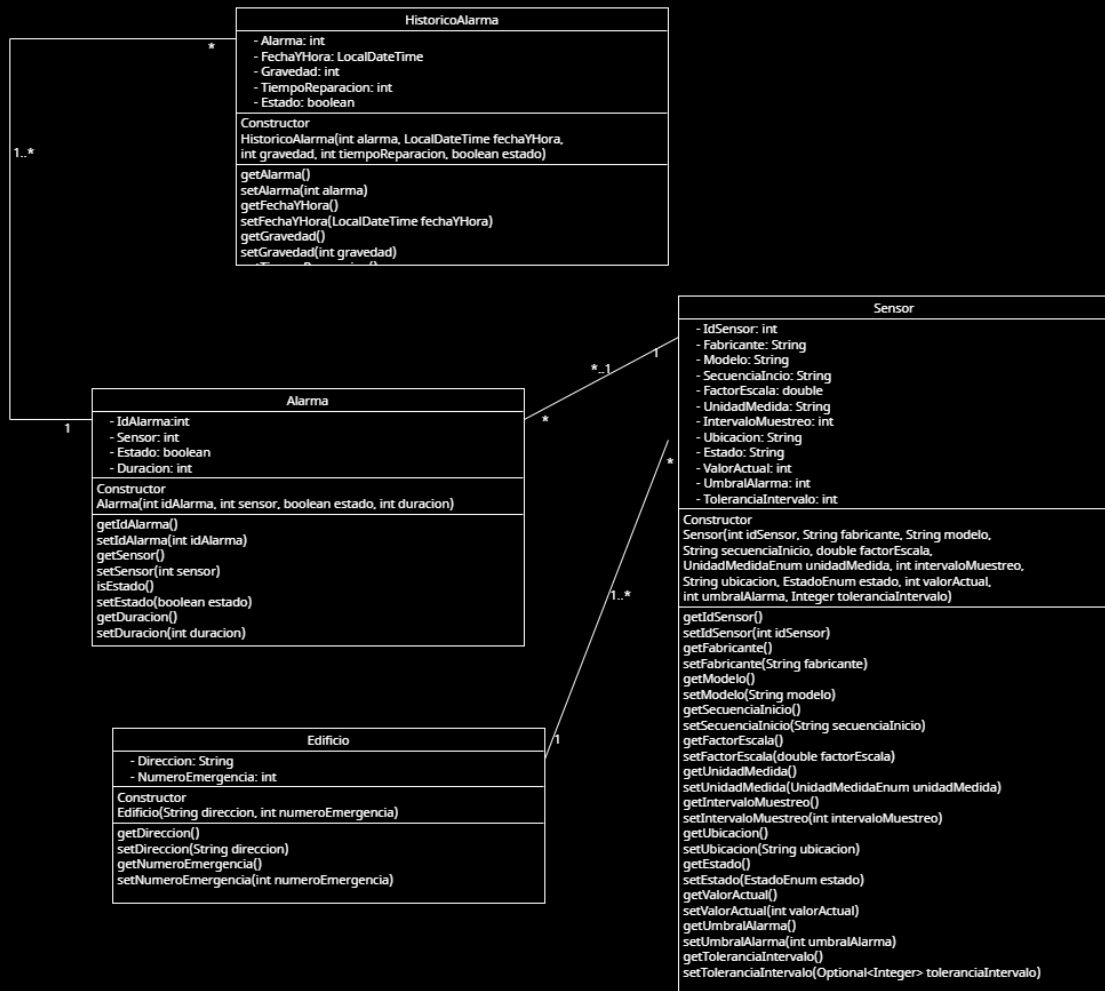
Clase Alarma:

Método	Descripción
Alarma(int idAlarma, int sensor, boolean estado, int duracion)	Constructor que inicializa el ID de la alarma, el sensor asociado, el estado y la duración de la alarma.
getIdAlarma()	Devuelve el ID de la alarma.
setIdAlarma(int idAlarma)	Establece el ID de la alarma.
getSensor()	Devuelve el ID del sensor asociado a la alarma.
setSensor(int sensor)	Establece el ID del sensor asociado a la alarma.
isEstado()	Devuelve el estado de la alarma (encendida o apagada).
setEstado(boolean estado)	Establece el estado de la alarma (encendida o apagada).
getDuracion()	Devuelve la duración de la alarma.
setDuracion(int duracion)	Establece la duración de la alarma.

Clase HistoricoAlarma:

Método	Descripción
HistoricoAlarma(int alarma, LocalDateTime fechaYHora, int gravedad, int tiempoReparacion, boolean estado)	Constructor que inicializa todos los atributos del historial de la alarma.
getAlarma()	Devuelve el ID de la alarma asociada al historial.
setAlarma(int alarma)	Establece el ID de la alarma asociada al historial.
getFechaYHora()	Devuelve la fecha y hora del evento en el historial.
setFechaYHora(LocalDateTime fechaYHora)	Establece la fecha y hora del evento en el historial.
getGravedad()	Devuelve la gravedad del evento registrado en el historial.
setGravedad(int gravedad)	Establece la gravedad del evento registrado en el historial.
getTiempoReparacion()	Devuelve el tiempo de reparación del evento registrado en el historial.
setTiempoReparacion(int tiempoReparacion)	Establece el tiempo de reparación del evento registrado en el historial.
isEstado()	Devuelve el estado del evento registrado en el historial (resuelto o no resuelto).
setEstado(boolean estado)	Establece el estado del evento registrado en el historial (resuelto o no resuelto).

- Crear el diagrama.



Conclusiones:

Personalmente no veo la lógica/utilidad de meter los métodos en el diagrama de clases, ya que eso lo vuelve mucho más engorroso y por lo tanto hace más difícil su lectura e interpretación. Ocultar los atributos de manera predeterminada, es una buena práctica que debería llevarse a cabo siempre que el lenguaje lo permita, por lo que es algo que va implícito o al menos debería.

En el segundo ejercicio he decidido añadirle el atributo `ToleranciaIntervalo`, en lugar de crear una nueva clase que extienda de `sensor`, ya que por los requisitos que se piden, no parece que la herencia vaya a ser útil para más de una clase hija, así que he añadido ese atributo con la idea de que ese atributo será `NULL` cuando se trate de un sensor normal, y tendrá valor cuando sea un `SensorCrítico`.

Respecto a la utilidad de estas técnicas, considero que es muy importante ya que permite al desarrollador encontrar rápidamente las relaciones entre clases y, las características de cada atributo. En mi caso, además tengo que añadir (en mi empresa) en la tabla de atributos datos y/o restricciones como cuales son únicos, id o clave, también enlazar mediante anclas las clases uniendo cada atributo que identifica uno o varios objetos de otra clase con la clase que representa. Aunque nunca incluimos los getter y setter en esa documentación.

Diagramas exportados desde <https://www.umletino.com/umletino.html> ambas aplicaciones propuestas para la creación de diagramas son muy similares, personalmente en el trabajo uso `draw.io`, aunque me gustaría sugerirte <https://dbdiagram.io/> que también es gratuita y permite crearlo todo usando pseudocódigo, creo que cuando te acostumbras es mucho más rápida y tiene más utilidades.